

# Getting Started

## Running the Universal Audio Plugin Template in Claude Code

*How to unpack the workflow files, point an IDE (VS Code, Google Antigravity, etc.) at the project, and run the exact first-session script — Phase 0 through the start of Phase 1B — using Claude Code.*

---

### Step 1 — Unpack the files into your project folder

Unpack the tarball directly into the folder where your plugin project will live. Do not leave the files sitting in a Downloads or temp folder — Claude Code discovers CLAUDE.md and .claude/skills/ based on the current working directory, so they need to live at the project root.

```
cd ~/DEV
mkdir PluginName
cd PluginName
tar -xzf ~/Downloads/plugin-workflow-package.tar.gz --strip-components=1
```

After this, CLAUDE.md, .claude/skills/, Docs/, and Tests/ should sit directly inside ~/DEV/PluginName/ — not nested inside another folder.

### Step 2 — Fill in the two documents everything else depends on

Do this in a text editor, before opening Claude Code. It's tedious form-filling, not something worth burning a conversation on.

1. Docs/PluginSpec.md — every field. This is the single document every skill reads from. Leaving it half-empty means every phase downstream will ask you the same questions back.
2. CLAUDE.md § 1 — the project identity table (plugin name, manufacturer code, JUCE path, project root, etc).

**Note:** Skip Docs/State/\*.md for now. Those are empty templates meant to fill in during Phase 1, not before.

### Step 3 — Open the IDE in that exact folder

In VS Code or Antigravity: File → Open Folder → select ~/DEV/PluginName. Then open the Claude Code panel or terminal from inside that workspace.

**Note:** If Claude Code's working directory isn't the project root, it will not find CLAUDE.md or .claude/skills/ at all, and the entire routing system will silently do nothing. This is the most common first-session failure.

## Step 4 — Run the first-session script

Send these messages one at a time, in order. Read each response before sending the next — don't batch them.

### Message 1 — Sanity check — confirms Claude Code can see your files

```
Read CLAUDE.md and confirm you have it. List the phase roadmap from § 3.
Then read Docs/PluginSpec.md and tell me which fields are still empty.
Do not begin any implementation.
```

*If this fails (Claude says it can't find the file): your working directory is wrong. Check with `pwd` in the Claude Code terminal — it must be the project root.*

### Message 2 — Phase 0 — toolchain check

```
Let's go through the Phase 0 checklist in CLAUDE.md § 6.
Check what you can verify yourself via the shell (cmake --version,
xcode-select -p, git --version, JUCE folder existence at the path I gave
in CLAUDE.md § 1). For anything you can't verify yourself, ask me directly.
```

Answer whatever it asks. If JUCE isn't installed yet, install it now before continuing — Phase 2 needs it.

### Message 3 — Close out Phase 0

```
Phase 0 complete. Update CLAUDE.md § 2 current phase to:
Phase 1 / NOT STARTED.
```

Small thing, but it matters — this is the one stateful line in the whole system, and it should be accurate from the start.

### Message 4 — Start Phase 1A — DSP research

```
Phase 1 starting. Load .claude/skills/dsp-research/SKILL.md.
Read Docs/PluginSpec.md § 2.2, 2.3, 2.7.
Begin Phase 1A — produce the DSP Design Document. Do not write any code.
```

Claude will produce the design document inline in chat. Read it carefully — push back on anything that doesn't match your sonic intent before approving. This is the cheapest point in the whole project to catch a wrong assumption.

### Message 5 — Approve and commit to state

Once you're happy with the design document:

```
Approved. Write this into Docs/State/DSP_Design.md, replacing the
placeholder content section by section. Append one line to
Docs/State/Changelog.md per the format in CLAUDE.md § 8.
```

### Message 6 — Phase 1B — architecture

```
Phase 1B starting. Load .claude/skills/plugin-architecture/SKILL.md.  
Read Docs/State/DSP_Design.md.  
Produce the Software Architecture Document. No code yet.
```

Same pattern as before — review, then approve, then:

```
Approved. Write this into Docs/State/Software_Architecture.md.  
Append to Docs/State/Changelog.md.  
Update CLAUDE.md § 2 current phase to: Phase 2 / NOT STARTED.
```

## Step 5 — The repeating rhythm for every phase after this

From here on, every phase follows the same three-message pattern. The phase-to-skill mapping always lives in CLAUDE.md § 3 if you forget which skill goes with which phase — you can ask Claude "what's next" and it will read the table itself.

3. Phase N starting. Load .claude/skills/[skill-name]/SKILL.md. Read [relevant Docs/State files].
4. Work through it, review, push back where needed.
5. Approved. Write into Docs/State/[Domain].md. Append to Changelog. Update CLAUDE.md § 2.

## One practical habit

Start a new Claude Code conversation per phase, not one continuous thread. The Docs/State/ files are what carry context between sessions — that's the whole point of replacing handoffs with them. A fresh conversation that reads Docs/State/\*.md will have cleaner context than a single long thread accumulating drift.

**Note:** Claude Code does not auto-load skill files just because they exist in .claude/skills/. You must explicitly tell it to read one the first time in a session ("Load .claude/skills/dsp-research/SKILL.md"). If a session feels like it's ignoring the rules, check whether the skill was actually read into context that conversation.